# SensorGatewaySoC

SoC para roteamento de mensagens entre sensores

Relatório técnico — Projeto 3: modelagem com SysML e AADL

Yuri e Luan

Outubro de 2026

## 1 Introdução

O SensorGatewaySoC é um SoC simplificado que concentra as mensagens de três sensores (temperatura, luminosidade e movimento), valida e classifica cada mensagem por prioridade, guarda temporariamente as que não podem ser enviadas de imediato e as encaminha a uma interface de saída, que pode representar UART, Ethernet, rádio ou CAN e não é implementada. Há dois desafios centrais: eventos críticos, como movimento detectado ou temperatura acima do limite, precisam chegar à saída antes das mensagens periódicas e dentro de um limite de latência; e o sistema precisa reagir de forma previsível quando a saída fica indisponível ou quando o buffer se esgota.

O trabalho segue o caminho do alto nível para a arquitetura embarcada. Em SysML (Papyrus 7.1, perfil SysML 1.6) estão os requisitos, os casos de uso, a estrutura (BDD e IBD) e o comportamento (atividade e máquina de estados). Em AADL (OSATE 2.20), a mesma solução é refinada em plataforma de execução, processos, threads, portas com filas, propriedades temporais e alocação do software ao hardware, o que permite analisar a latência do fluxo fim a fim. A Tabela 1 relaciona os artefatos exigidos com as seções deste relatório.

**Tabela 1:** Artefatos produzidos e onde estão neste relatório.

| **Artefato** | **Conteúdo** | **Onde** |
|---|---|---|
| Documento de visão | Problema, usuários, contexto e objetivos | Anexo A |
| Requisitos SysML | 12 requisitos: 9 funcionais e 3 não funcionais | Seção 2 e Anexo D |
| Casos de uso | 3 casos de uso principais e 1 de exceção; 5 atores | Seção 2 e Anexo B |
| Estrutura SysML | BDD com 8 blocos e IBD com 13 conectores | Seção 3 |
| Comportamento SysML | Diagrama de atividade e máquina de estados com 6 estados | Seção 4 |
| Arquitetura AADL | 1 *system*, 3 *process*, 5 *thread*, 1 *processor*, 1 *memory*, 1 *bus* e 4 *device* | Seção 5 e Anexo E |
| Fluxo AADL | Fluxo fim a fim `e2e_critical` e análise de latência | Seção 7 |
| Testes | 5 casos de teste, 3 deles de exceção ou falha | Seção 8 e Anexo C |
| Rastreabilidade | Matriz requisito–caso de uso–bloco SysML–elemento AADL–teste | Seção 8 |

## 2 Requisitos e casos de uso

Os dez requisitos sugeridos no enunciado foram mantidos (RQ01 a RQ10) e complementados por dois requisitos não funcionais que tornam mensuráveis as qualidades pedidas: RQ11 fixa o orçamento de latência do caminho mais sensível ao tempo, do sensor de movimento até a saída, e RQ12 fixa a capacidade do buffer. O RQ03 é classificado como não funcional porque não acrescenta uma função: ele restringe a ordem em que as mensagens são atendidas. A Tabela 2 lista os requisitos; o diagrama de requisitos está no Anexo D.

O diagrama de casos de uso (Figura 1) tem cinco atores: os três sensores, o Sistema Externo de Supervisão, que recebe as mensagens, e o Operador, que acompanha o estado da saída e do buffer. UC01 (transmitir leitura periódica), UC02 (transmitir evento crítico) e UC03 (tratar saída indisponível) são os casos de uso principais. O UC04 (tratar mensagem inválida) cobre o cenário de exceção do enunciado: é alcançado a partir do passo de validação do UC01 e do UC02 e está associado à Supervisão, a parte interessada no seu resultado, já que nenhuma mensagem inválida deve chegar a ela. Como em todo diagrama de casos de uso, quem executa cada caso de uso é o próprio SensorGatewaySoC; os atores são as entidades externas que interagem com ele. As especificações completas, com pré-condições, fluxos principais e alternativos, estão no Anexo B.

**Tabela 2:** Requisitos do SensorGatewaySoC (F: funcional; NF: não funcional).

| ID | Tipo | Texto |
|---|---|---|
| RQ01 | F | O sistema deve receber mensagens de pelo menos três tipos de sensores (temperatura, luminosidade, movimento). |
| RQ02 | F | Cada mensagem deve conter identificador de origem, tipo, valor e prioridade. |
| RQ03 | NF | Eventos críticos devem ser encaminhados antes de mensagens periódicas não críticas. |
| RQ04 | F | O sistema deve armazenar temporariamente mensagens que não possam ser transmitidas imediatamente. |
| RQ05 | F | O sistema deve indicar condição de buffer cheio. |
| RQ06 | F | O sistema deve registrar o número de mensagens processadas por sensor. |
| RQ07 | F | O sistema deve descartar ou marcar mensagens inválidas. |
| RQ08 | F | O sistema deve suportar ao menos dois níveis de prioridade. |
| RQ09 | F | O sistema deve transmitir mensagens em formato padronizado. |
| RQ10 | F | O sistema deve entrar em modo de degradação se a interface de saída estiver indisponível. |
| RQ11 | NF | O evento crítico de movimento deve ser transmitido à saída em no máximo 100 ms. |
| RQ12 | NF | O buffer deve ter capacidade limitada de 4 mensagens críticas e 16 mensagens periódicas. |

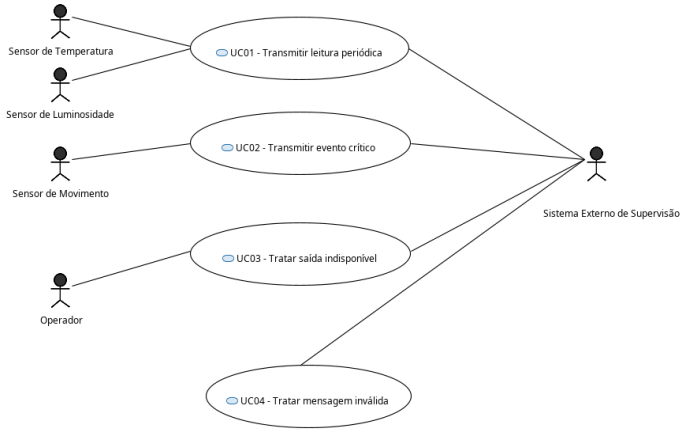



**Figura 1:** Diagrama de casos de uso.

Cada caso de uso se liga aos requisitos que detalha por uma relação «refine». A especificação do SysML 1.6 [1] define «refine» como a relação em que um elemento do modelo detalha um requisito textual, e cita o caso de uso como exemplo típico. Já «satisfy» liga um elemento de *projeto*, como um bloco, ao requisito que ele cumpre. Por isso, nos diagramas, os casos de uso refinam os requisitos, e a satisfação é atribuída aos blocos na matriz de rastreabilidade (Seção 8). O diagrama dessas relações está no Anexo D.

# 3 Estrutura em SysML

O BDD (Figura 2) decompõe o `SensorGatewaySoC` nos sete blocos pedidos no enunciado, totalizando os oito blocos permitidos. A composição (losango cheio) indica que as partes só existem dentro do SoC. Cada bloco tem uma responsabilidade: `SensorInterface` recebe e valida as mensagens dos sensores; `MessageFormatter` classifica cada mensagem e preenche sua prioridade; `MessageBuffer` retém as mensagens em duas filas, crítica e periódica; `PriorityArbiter` escolhe a próxima mensagem a transmitir; `OutputInterface` representa o canal de saída e informa se ele está ativo; `StatisticsManager` mantém os contadores por sensor; `ConfigurationMemory` guarda o limite de temperatura que separa as leituras normais das críticas.

O IBD (Figura 3) mostra como as partes trocam mensagens. O caminho principal é `sensorinterface` → `messageformatter` → `messagebuffer` → `priorityarbiter` → `outputinterface`, com canais separados para mensagens críticas (`crit`) e periódicas (`per`) a partir do classificador. Três ligações merecem destaque. O árbitro devolve ao buffer um *crédito* por mensagem transmitida (`out_ccred` e `out_pcred`), o que permite ao buffer saber quantas posições estão livres. O sinal `out_full` do buffer segue para a saída e para a estatística. A saída informa a queda e a volta do canal ao árbitro (`out_down` e `out_up`). As portas não foram tipadas no SysML: o formato da mensagem é definido no AADL (`SG_Types`). O acesso de leitura do classificador à `configurationmemory` também é detalhado no AADL.

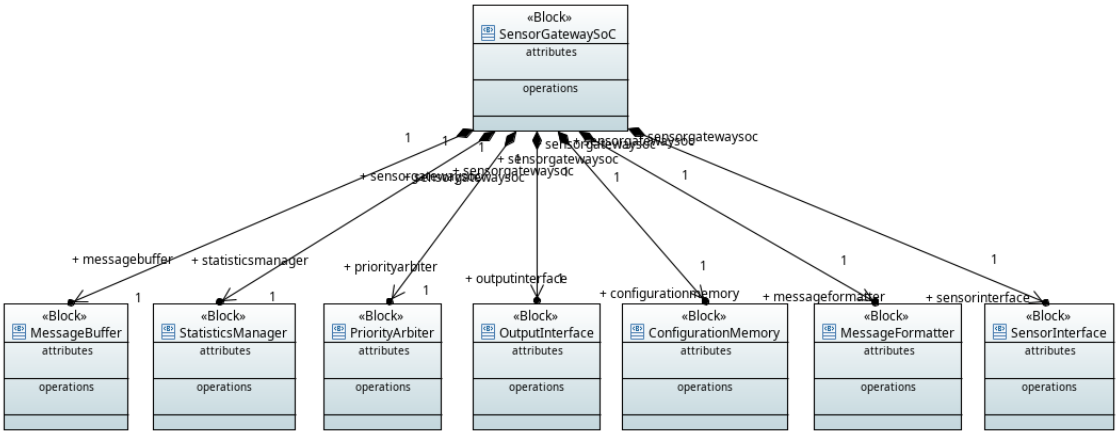



**Figura 2:** BDD: composição do `SensorGatewaySoC`.

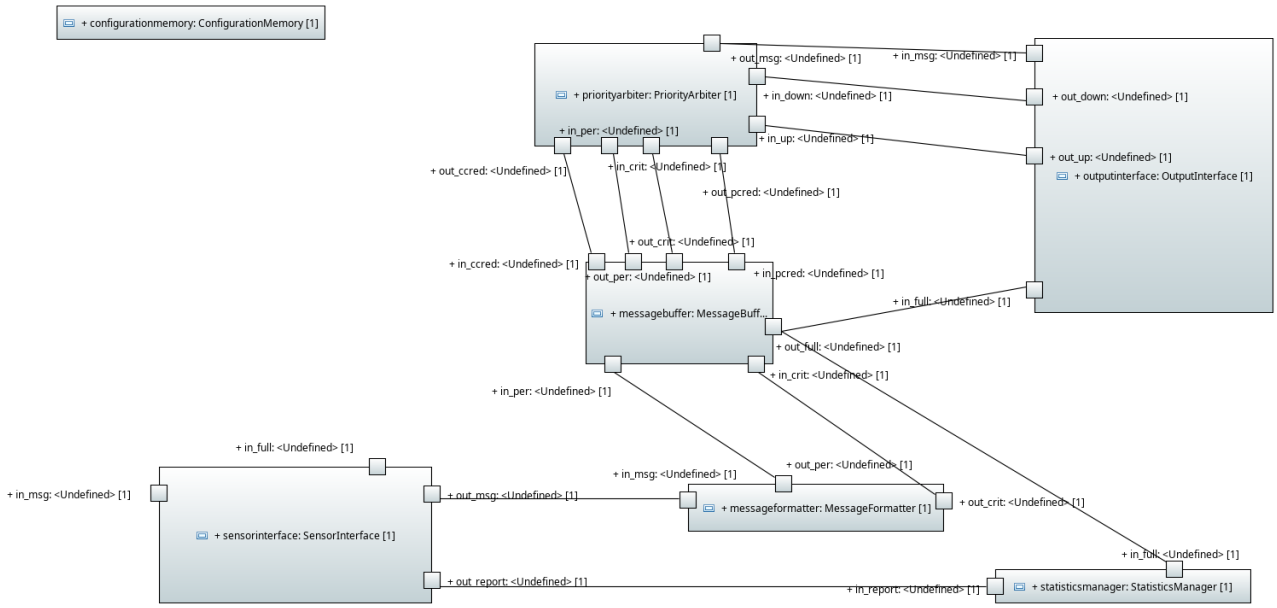



**Figura 3:** IBD: fluxo de mensagens, créditos e sinais de estado entre as partes.

# 4 Comportamento em SysML

O diagrama de atividade (Figura 4) segue a sequência pedida: receber → validar → classificar → enfileirar → selecionar → transmitir. A primeira decisão separa as mensagens inválidas, que são descartadas e contabilizadas na estatística (RQ06, RQ07). As válidas são classificadas e seguem para a fila crítica ou para a periódica (RQ03, RQ08). Antes da seleção, a capacidade da fila é verificada: se ela estiver cheia, o buffer cheio é sinalizado (RQ05). Por fim, a mensagem selecionada é transmitida se o enlace estiver ativo, ou retida em caso contrário (RQ04, RQ10).

A máquina de estados do gerenciador de mensagens (Figura 5) tem os seis estados permitidos: Idle, Receiving, Buffered, Transmitting, OutputUnavailable e BufferFull. O enunciado sugere também um estado Classifying. Como a classificação não espera nenhum evento externo, ela foi incorporada ao estado Receiving, o que mantém a máquina dentro do limite de seis estados.

- **Operação normal:** Idle → Receiving (`mensagem_recebida`) → Buffered (`mensagem_enfileirada`) → Transmitting (`árbitroSeleciona_enlaceAtivo`). De Transmitting, a máquina volta a Buffered se ainda há mensagens (`mensagem_transmitida`) ou a Idle se as filas esvaziaram (`filas_vazias`). Uma mensagem inválida leva de Receiving direto a Idle (`mensagem_inválida`).
- **Degradação:** `link_down` leva qualquer estado de operação a OutputUnavailable, e `link_up` retorna a Buffered (RQ10).
- **Esgotamento:** `sem_crédito` leva de Buffered ou de OutputUnavailable a BufferFull (RQ05), e

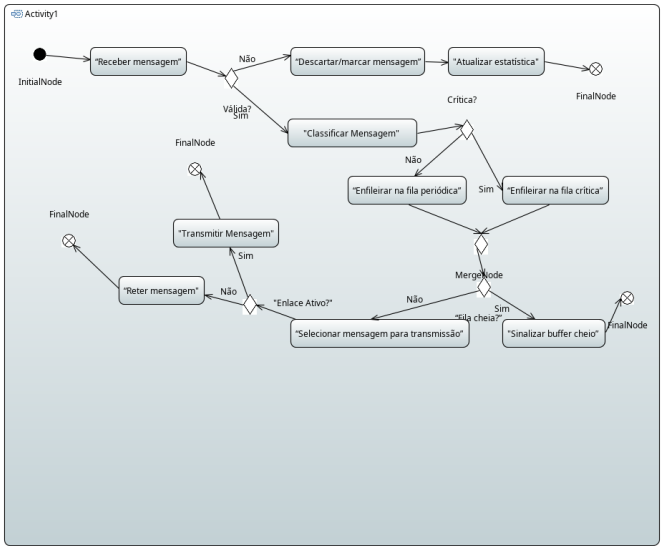



**Figura 4:** Diagrama de atividade do processamento de uma mensagem.

`credito_disponivel` retorna a Buffered.

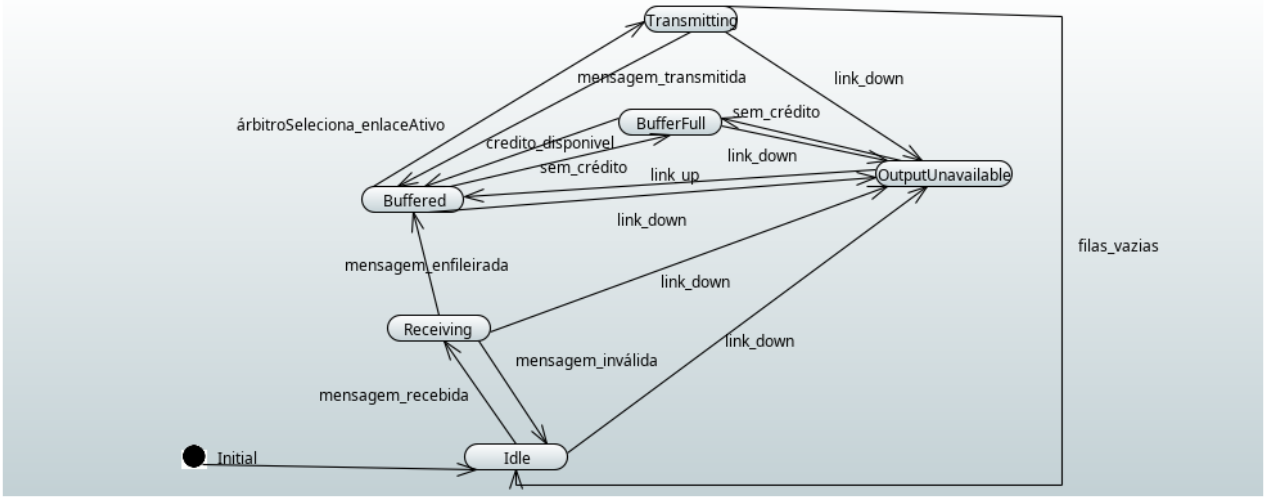



**Figura 5:** Máquina de estados do gerenciador de mensagens.

# 5 Arquitetura em AADL

O modelo AADL [2] está dividido em quatro pacotes. `SG_Types` define os dados: a mensagem `Msg` de 8 bytes (origem, tipo, prioridade, um byte de alinhamento e o valor de 32 bits), o relatório `RxReport` de 2 bytes e a configuração `Config` de 4 bytes. `SG_Platform` define a plataforma: processador `Cpu` com escalonamento RMS, memória `Ram` de 64 KB, barramento `SocBus` e os dispositivos dos três sensores e da interface de saída. `SG_Software` define as cinco threads e os três processos. `SG_System` integra tudo. O pacote de software não importa o de plataforma: a alocação do software ao hardware é decidida apenas no `SG_System`, pelas propriedades de *binding*. A Figura 6 mostra a instância do sistema.

Os três processos seguem o particionamento do enunciado: `acquisition` (aquisição e validação), `classification` (classificação e armazenamento) e `transmission`. A `statistics` fica no processo de aquisição porque recebe um relatório a cada mensagem do `receiver`, e assim a conexão mais frequente fica dentro de um mesmo processo. A Tabela 3 resume as propriedades temporais das threads. Para as esporádicas, o período é o intervalo mínimo entre dois despachos. As prioridades seguem o *Rate Monotonic*: período menor, prioridade maior.

As principais decisões de projeto foram estas:

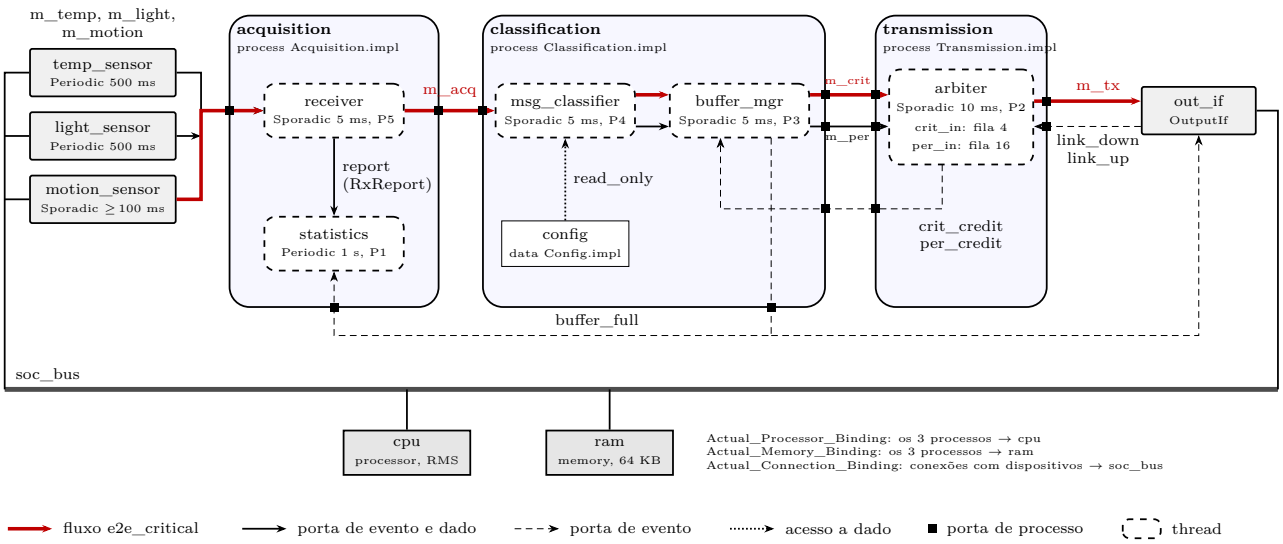



**Figura 6:** Instância de `SensorGatewaySoC.impl`: processos, threads, dispositivos, conexões e fluxo fim a fim. P indica a prioridade da thread (maior número, maior prioridade).

**Tabela 3:** Threads do modelo AADL. $C$ é o tempo de execução (`Compute_Execution_Time`), $T$ o período e $D$ o *deadline*.

| Thread | Processo | Despacho | $T$ (ms) | $C$ (ms) | $D$ (ms) | Prior. | $C_{\max}/T$ |
|---|---|---|---|---|---|---|---|
| `receiver` | acquisition | Sporadic | 5 | 0,2–0,5 | 5 | 5 | 0,100 |
| `msg_classifier` | classification | Sporadic | 5 | 0,2–0,5 | 5 | 4 | 0,100 |
| `buffer_mgr` | classification | Sporadic | 5 | 0,1–0,3 | 5 | 3 | 0,060 |
| `arbiter` | transmission | Sporadic | 10 | 1–2 | 10 | 2 | 0,200 |
| `statistics` | acquisition | Periodic | 1000 | 0,5–1 | 1000 | 1 | 0,001 |
| Utilização total do processador | | | | | | | 0,461 |

- **Filas como portas.** As duas filas do buffer são as portas de entrada do `arbiter`: `crit_in` com `Queue_Size` 4 e `per_in` com 16 (RQ04, RQ12). Em AADL, armazenamento entre threads é uma propriedade das portas de evento e dado.
- **Prioridade estrita.** A cada despacho, o `arbiter` atende a fila não vazia de maior `Urgency`: 2 para `crit_in` e 1 para `per_in`. Assim, uma mensagem crítica pendente é sempre transmitida antes de qualquer periódica (RQ03, RQ08).
- **Controle por créditos.** Uma thread não enxerga a ocupação da fila de outra thread. O `buffer_mgr` começa com 4 créditos críticos e 16 periódicos, gasta um a cada mensagem encaminhada e recebe um de volta (`crit_credit`, `per_credit`) a cada mensagem que o `arbiter` transmite. Sem crédito, a fila está cheia: o `buffer_mgr` dispara `buffer_full` e descarta a mensagem que acabou de chegar, contabilizando o descarte (RQ05). Por isso as filas do `arbiter` nunca transbordam. Mesmo assim, `crit_in` usa `Overflow_Handling_Protocol => Error`, para que uma mensagem crítica nunca desapareça em silêncio, e `per_in` usa `DropOldest`, já que uma leitura nova vale mais que uma antiga.
- **Degradação.** As portas `link_down` e `link_up` do `arbiter` têm `Urgency` 3, acima das filas de mensagens. Assim, uma mudança no estado do canal é tratada antes de qualquer mensagem pendente. Com o canal caído, o `arbiter` retém as mensagens, e o restante do pipeline continua operando (RQ10).
- **Estatística em lote.** A `statistics` é periódica (1 s) e de baixa prioridade. Ela retira de uma só vez todos os relatórios acumulados (`Dequeue_Protocol => AllItems`). A fila de 16 posições cobre a taxa máxima de chegada, de 14 mensagens/s (RQ06).
- **Alocação.** Os três processos estão ligados ao `cpu` e à `ram`. Apenas as conexões que envolvem dispositivos passam pelo `soc_bus`; entre processos na mesma CPU, a comunicação se dá pela memória.

# 6 Refinamento de SysML para AADL

A correspondência entre os dois modelos não é de um para um, e não deveria ser: o SysML diz *o que* cada parte faz, e o AADL diz *onde* e *quando* isso executa. A Tabela 4 mostra como cada bloco foi refinado.

**Tabela 4:** Refinamento dos blocos SysML em elementos AADL.

| Bloco SysML | Elementos AADL | Tipo de refinamento |
|---|---|---|
| SensorGatewaySoC | `system SensorGatewaySoC.impl`; `cpu`, `ram`, `soc_bus` | O AADL acrescenta a plataforma de execução e a alocação, ausentes no SysML. |
| SensorInterface | devices `temp_sensor`, `light_sensor`, `motion_sensor`; thread `receiver` | Divisão: o lado físico vira dispositivo, e a recepção e validação viram thread. |
| MessageFormatter | thread `msg_classifier`; dado `Msg.impl` | A classificação vira thread; o formato padronizado vira tipo de dado de 8 bytes. |
| MessageBuffer | thread `buffer_mgr`; filas `crit_in` e `per_in` do `arbiter` | Absorção: o armazenamento vira propriedade de portas, e o controle de ocupação vira thread. |
| PriorityArbiter | thread `arbiter`; propriedade `Urgency` | A política de prioridade vira propriedade das portas. |
| OutputInterface | device `out_if` | Mantido como dispositivo na fronteira, com os eventos de estado do canal. |
| StatisticsManager | thread `statistics`; dado `RxReport.impl` | Thread periódica de baixa prioridade. |
| ConfigurationMemory | dado `config` (`Config.impl`); memória `ram` | O conteúdo vira componente de dado com acesso só de leitura; o armazenamento físico é a memória. |

Três tipos de refinamento aparecem. O primeiro é a *divisão*: um bloco SysML se torna mais de um elemento AADL de categorias diferentes, como a `SensorInterface`, que vira dispositivos e uma thread. O segundo é a *absorção*: um bloco deixa de existir como componente próprio, como o `MessageBuffer`, cujas filas passam a ser portas de outra thread. O terceiro é o *acréscimo*: elementos que não fazem sentido no nível do SysML, como processador, barramento, *bindings*, períodos, tempos de execução e tamanhos de dado. Mesmo com essas mudanças, cada um dos 13 conectores do IBD tem uma conexão AADL correspondente. Por exemplo, os créditos `out_ccred` e `out_pcred` correspondem a `e_ccred` e `e_pcred`, e o sinal `out_full`, que no IBD tem dois conectores, corresponde a `e_full_stat` e `e_full_out`.

# 7 Fluxo fim a fim e análises

O fluxo `e2e_critical` segue o evento crítico do sensor de movimento até a saída, alternando fluxos de componente e conexões, conforme o caminho pedido no enunciado (sensor → recepção → classificação → buffer → arbitragem → saída):

`motion_sensor.src` → `m_motion` → `acquisition.fp` → `m_acq` → `classification.fp_crit` → `m_crit` → `transmission.fp_crit` → `m_tx` → `out_if.snk`

O fluxo declara a propriedade `Latency` com o intervalo de 0 a 100 ms, que é o RQ11. Dentro dos processos, `classification.fp_crit` passa pelo `msg_classifier` e pelo `buffer_mgr`, e `transmission.fp_crit` passa pela fila crítica do `arbiter`.

**Latência.** Para uma thread despachada por evento com fila, a análise de latência de fluxo do OSATE [3] conta, no pior caso, um período de espera por posição da fila ocupada à frente da mensagem, ou seja, $T \times$ `Queue_Size`. A esse termo soma o tempo de processamento. Foram usados dois modos de análise. No modo SS-PE-ET-FQ-EQL, o pior caso de processamento é o $C$ máximo, e o resultado é de 91,5 a 95,3 ms, dentro do limite. No modo AS-PE-DL-EQ-DQL, o pior caso de processamento é o *deadline* de cada thread, e o total chega a 117,0 ms, o que o OSATE aponta como violação do RQ11. A Tabela 5 decompõe os dois resultados.

As filas respondem por 90 ms dos 95,3 ms. O maior termo é a fila crítica do `arbiter`: cada posição a mais custa 10 ms no pior caso. Isso expõe a troca entre capacidade (RQ12) e latência (RQ11). O modo DL é pessimista porque supõe que cada thread usa todo o seu *deadline*. Um limite mais justo é o tempo de resposta no pior caso $R_i$, dado pela análise de tempo de resposta (RTA) do escalonamento de prioridade fixa [5]: $R_i$ é o

**Tabela 5:** Contribuições para a latência máxima de `e2e_critical`, em ms. As colunas ET e DL reproduzem os relatórios do OSATE; a coluna $R$ usa o tempo de resposta pelo RTA no lugar do processamento.

| Elemento | Fila ($T\times$ Queue_Size) | ET: $C_{\max}$ | DL: $D$ | RTA: $R$ |
|---|---|---|---|---|
| `motion_sensor` (origem) | — | 1,0 | 1,0 | 1,0 |
| `receiver` | $5 \times 4 = 20$ | 0,5 | 5,0 | 0,5 |
| `msg_classifier` | $5 \times 4 = 20$ | 0,5 | 5,0 | 1,0 |
| `buffer_mgr` | $5 \times 2 = 10$ | 0,3 | 5,0 | 1,3 |
| `arbiter` | $10 \times 4 = 40$ | 2,0 | 10,0 | 3,3 |
| `out_if` (destino) | — | 1,0 | 1,0 | 1,0 |
| **Total** (fila + processamento) | 90 | **95,3** | **117,0** | **98,1** |

menor valor que satisfaz

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j,$$

em que $hp(i)$ é o conjunto das threads de prioridade maior que a de $i$. Com os valores da Tabela 3, obtêm-se 0,5 ms para o `receiver`, 1,0 ms para o `msg_classifier`, 1,3 ms para o `buffer_mgr` e 3,3 ms para o `arbiter`. Com as mesmas hipóteses de filas cheias do OSATE, a latência máxima fica em 98,1 ms, abaixo dos 100 ms. O RQ11 é atendido, mas com margem pequena, de 1,9 ms; reduzir a fila crítica do `arbiter` ou o seu período é o primeiro ajuste se o orçamento precisar de folga.

**Escalonabilidade.** A utilização do processador é $U = \sum_i C_i/T_i = 0{,}461$. Para $n = 5$ threads, o limite de Liu e Layland [4] para o RMS é $n(2^{1/n} - 1) \approx 0{,}743$. Como $U$ fica abaixo desse limite, todas as threads cumprem seus *deadlines*, o que também confirma os valores de $R_i$ acima, todos menores que os respectivos $D_i$.

**Capacidade.** Os sensores geram no máximo 14 mensagens/s: 2 de temperatura, 2 de luminosidade e 10 de movimento. O `arbiter` transmite até 100 mensagens/s, então as filas não crescem em operação normal. Com a saída indisponível, a fila crítica, que recebe até 12 mensagens/s, enche em cerca de 0,33 s; a periódica, que recebe até 4 mensagens/s, em pelo menos 4 s. Os dados em fila ocupam cerca de 292 bytes, muito abaixo dos 64 KB da `ram`.

# 8 Verificação e rastreabilidade

Os cinco casos de teste (Tabela 6) são cenários simulados sobre o modelo, como permite o enunciado; três deles são de exceção ou falha (TC03, TC04 e TC05). Cada teste indica o método de verificação. Na *análise*, a evidência é produzida por ferramenta: a validação do OSATE, sem erros, confere os tipos das portas em cada conexão e a coerência dos tamanhos de dado, e a análise de latência verifica o fluxo. Na *inspeção*, o cenário é percorrido no modelo, conferindo que cada elemento necessário existe, tem as propriedades exigidas e está conectado. O caminho de cada teste pelo AADL e pela máquina de estados está no Anexo C.

**Tabela 6:** Casos de teste.

| Teste | Estímulo | Resultado esperado | Método | Verifica |
|---|---|---|---|---|
| TC01 | Mensagem periódica válida de temperatura | Mensagem é classificada, armazenada e encaminhada | Inspeção e análise | RQ01, RQ02, RQ04, RQ06, RQ09 |
| TC02 | Temperatura e evento crítico de movimento simultâneos | Evento crítico recebe prioridade de transmissão; latência $\leq 100$ ms | Análise e inspeção | RQ01, RQ03, RQ08, RQ11 |
| TC03 | Interface de saída indisponível | Mensagens são retidas no buffer e o sistema entra em OutputUnavailable | Inspeção | RQ04, RQ10 |
| TC04 | Buffer atinge a capacidade máxima | Sistema sinaliza BufferFull e descarta a mensagem recém-chegada, contabilizando-a | Inspeção | RQ05, RQ12 |
| TC05 | Mensagem com formato inválido | Mensagem é descartada; estatística é atualizada | Inspeção | RQ01, RQ06, RQ07 |

A matriz de rastreabilidade (Tabela 7) liga cada requisito ao caso de uso que o refina, ao bloco SysML que o satisfaz, ao elemento AADL que o implementa e ao teste que o verifica. Todos os doze requisitos têm pelo menos um teste. O RQ01 é verificado pelo conjunto, já que os três tipos de sensor aparecem como origem em TC01 (temperatura), TC02 (movimento) e TC05 (luminosidade).

**Tabela 7:** Matriz de rastreabilidade requisito–caso de uso–bloco SysML–elemento AADL–teste.

| Req. | Tipo | Caso de uso | Bloco SysML | Elemento AADL | Teste |
|---|---|---|---|---|---|
| RQ01 | F | UC01, UC02 | SensorInterface | devices `temp_sensor`, `light_sensor`, `motion_sensor`; thread `Receiver`; conexões `m_temp`, `m_light`, `m_motion` | TC01, TC02, TC05 |
| RQ02 | F | UC01, UC02 | MessageFormatter | `Msg.impl` com `source_id`, `msg_type`, `priority` e `value` | TC01 |
| RQ03 | NF | UC02 | PriorityArbiter | thread `Arbiter`: `crit_in` com `Urgency` 2 e `per_in` com `Urgency` 1; canais separados `crit_*` e `per_*` | TC02 |
| RQ04 | F | UC01, UC03 | MessageBuffer | filas `crit_in` e `per_in` do `Arbiter`; thread `BufferMgr` | TC01, TC03 |
| RQ05 | F | UC03 | MessageBuffer | porta `BufferMgr.buffer_full`; conexões `e_full_stat` e `e_full_out` | TC04 |
| RQ06 | F | UC01, UC04 | StatisticsManager | `RxReport.impl`; thread `Statistics` (`report_in`, fila 16, `AllItems`) | TC01, TC05 |
| RQ07 | F | UC04 | SensorInterface | thread `Receiver`: só mensagens válidas seguem por `msg_out` | TC05 |
| RQ08 | F | UC02 | MessageFormatter | campo `priority`; thread `MsgClassifier` (`crit_out`, `per_out`); `Config.impl` | TC02 |
| RQ09 | F | UC01 | MessageFormatter | todas as portas de mensagem tipadas com `Msg.impl` (`Data_Size` = 8 Bytes) | TC01 |
| RQ10 | F | UC03 | PriorityArbiter, OutputInterface | `OutputIf.link_down` e `link_up` → `Arbiter` (`Urgency` 3); conexões `e_down` e `e_up` | TC03 |
| RQ11 | NF | UC02 | SensorGatewaySoC | fluxo `e2e_critical` com `Latency` 0..100 ms; latência máxima de 95,3 ms (ET) e 98,1 ms (RTA) | TC02 |
| RQ12 | NF | UC03 | MessageBuffer | `Arbiter.crit_in` com `Queue_Size` 4 e `per_in` com 16; créditos do `BufferMgr` | TC04 |

# 9 Conclusão

O modelo responde às perguntas que o enunciado coloca. A comunicação entre sensores e saída é organizada como um pipeline de três processos, com canais separados para mensagens críticas e periódicas. A prioridade é tratada por filas distintas no árbitro, com `Urgency` maior para a fila crítica. A indisponibilidade da saída é tratada por eventos de maior urgência que suspendem a transmissão sem interromper a recepção. O esgotamento do buffer é controlado por créditos, que sinalizam `buffer_full` e aplicam uma política de descarte definida, sem perder em silêncio as mensagens já aceitas. A rastreabilidade liga cada requisito a um caso de uso, a um bloco, a elementos AADL e a um teste.

Algumas limitações ficam registradas. Os testes são cenários verificados por inspeção e análise, não por execução: a lógica interna das threads (validação, classificação e créditos) está descrita nos comentários do modelo e no comportamento SysML, mas não foi formalizada. Uma extensão natural seria descrevê-la com o *Behavior Annex* do AADL. A margem de latência do RQ11 é pequena e depende dos tempos de execução declarados. As portas do IBD não foram tipadas, e o acesso à configuração só aparece no AADL.

# Referências


[1] OBJECT MANAGEMENT GROUP. *OMG Systems Modeling Language (OMG SysML), version 1.6*. 2019.

[2] SAE INTERNATIONAL. *AS5506C: Architecture Analysis & Design Language (AADL)*. 2017.

[3] FEILER, P. H.; GLUCH, D. P. *Model-Based Engineering with AADL*. Addison-Wesley, 2012.

[4] LIU, C. L.; LAYLAND, J. W. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, v. 20, n. 1, p. 46–61, 1973.

[5] JOSEPH, M.; PANDYA, P. Finding response times in a real-time system. *The Computer Journal*, v. 29, n. 5, p. 390–395, 1986.

# Anexo A Documento de visão

## Problema

Sistemas embarcados que integram múltiplos sensores — por exemplo, temperatura, luminosidade e movimento — precisam encaminhar suas leituras a um sistema externo de supervisão de forma organizada e confiável. Dois desafios centrais surgem nesse cenário: (1) eventos críticos, como a detecção de movimento ou uma leitura de temperatura fora da faixa esperada, precisam chegar ao destino com prioridade sobre mensagens periódicas de rotina, dentro de um limite de tempo aceitável; e (2) o sistema precisa continuar operando de forma previsível quando a interface de saída fica temporariamente indisponível, retendo as mensagens enquanto houver espaço de armazenamento e, quando ele se esgota, descartando de forma controlada e sinalizada. Além disso, leituras inválidas não podem contaminar o fluxo entregue à supervisão. Um agregador simples, que trata todas as mensagens com a mesma prioridade e não prevê a queda do canal de saída, não resolve esses desafios.

## Usuários

O sistema interage com os seguintes atores:

- **Sensores** de temperatura, luminosidade e movimento, que geram as mensagens de entrada;
- **Sistema externo de supervisão**, que recebe as mensagens processadas e consome os dados agregados;
- **Operador/administrador**, que acompanha o estado do gateway (estatísticas, condição de buffer, disponibilidade da saída).

## Contexto

O sistema é modelado como um SoC (*System on Chip*) embarcado de recursos limitados: um único processador, memória restrita para configuração e buffer, e um barramento interno que interliga o processador, a memória e as interfaces dos sensores e de saída. Ele opera de forma contínua, recebendo mensagens de três tipos de sensores que produzem de forma independente e em ritmos diferentes, e precisa responder a eventos críticos em tempo real, respeitando um orçamento de latência fim a fim definido para o caminho mais sensível ao tempo (evento de movimento até a saída). Ficam fora do escopo a implementação da interface física de saída (UART, Ethernet, rádio ou CAN), o projeto dos sensores e qualquer implementação em hardware. O projeto é desenvolvido como exercício acadêmico de modelagem arquitetural, combinando SysML (visão de alto nível, requisitos e comportamento) e AADL (arquitetura embarcada analisável).

## Objetivos

- Receber e padronizar mensagens de ao menos três tipos de sensores;
- Detectar e descartar mensagens inválidas, sem que afetem o encaminhamento das demais;
- Classificar cada mensagem por tipo e prioridade, distinguindo eventos críticos de leituras periódicas;
- Armazenar temporariamente mensagens que não podem ser transmitidas de imediato, com capacidade de buffer limitada, sinalização de buffer cheio e política de descarte definida;
- Priorizar o encaminhamento de eventos críticos sobre mensagens periódicas não críticas, dentro de um limite de latência fim a fim;
- Entrar em modo de degradação quando a interface de saída estiver indisponível, retendo as mensagens até o limite do buffer e retomando a transmissão quando ela voltar;
- Registrar estatísticas de mensagens processadas por sensor, para fins de monitoramento e diagnóstico.

# Anexo B Especificação dos casos de uso

## UC01 – Transmitir leitura periódica

| | |
|---|---|
| **Atores** | Primários: Sensor de Temperatura, Sensor de Luminosidade. Secundário: Sistema Externo de Supervisão. |
| **Pré-condição** | Sistema em operação normal; interface de saída disponível. |
| **Gatilho** | Fim do período de amostragem do sensor (500 ms). |
| **Fluxo principal** | 1. O sensor envia uma leitura com origem, tipo = leitura e valor. [**RQ01, RQ02**]<br>2. O sistema verifica se os campos da mensagem estão nas faixas válidas.<br>3. O sistema registra a recepção na estatística do sensor de origem. [**RQ06**]<br>4. O sistema classifica a leitura como periódica: luminosidade, ou temperatura dentro do limite configurado. [**RQ02**]<br>5. O sistema mantém a mensagem na fila periódica até que a saída possa transmiti-la. [**RQ04**]<br>6. O sistema transmite a mensagem à Supervisão no formato padronizado de 8 bytes. [**RQ09**] |
| **Fluxos alternativos** | 2a. A mensagem é inválida: segue o UC04.<br>4a. A temperatura está acima do limite: a leitura é classificada como crítica e segue o UC02 a partir do passo 4.<br>5a. A saída está indisponível: segue o UC03. |
| **Pós-condição** | A Supervisão recebeu a leitura; o contador do sensor de origem foi incrementado. |

## UC02 – Transmitir evento crítico

| | |
|---|---|
| **Atores** | Primário: Sensor de Movimento. Secundário: Sistema Externo de Supervisão. A leitura de temperatura acima do limite também segue este caso de uso, a partir do passo 4, pelo fluxo alternativo 4a do UC01. |
| **Pré-condição** | Sistema em operação normal; interface de saída disponível. |
| **Gatilho** | Detecção de movimento (no máximo um evento a cada 100 ms) ou leitura de temperatura acima do limite configurado (UC01, fluxo 4a). |
| **Fluxo principal** | 1. O sensor envia a mensagem; no caso do movimento, com tipo = evento. [**RQ01, RQ02**]<br>2. O sistema valida a mensagem e registra a recepção na estatística.<br>3. O sistema classifica a mensagem como crítica, distinguindo-a das periódicas. [**RQ08**]<br>4. O sistema coloca a mensagem na fila crítica.<br>5. Na transmissão seguinte, o sistema seleciona a mensagem crítica antes de qualquer mensagem periódica pendente. [**RQ03**]<br>6. O sistema transmite a mensagem à Supervisão. Do envio pelo sensor de movimento até a entrega decorrem no máximo 100 ms. [**RQ11**] |
| **Fluxos alternativos** | 2a. A mensagem é inválida: segue o UC04.<br>5a. A saída está indisponível: segue o UC03; a mensagem crítica é a primeira a ser transmitida quando a saída voltar. |
| **Pós-condição** | O evento crítico foi entregue antes das mensagens periódicas que aguardavam. |

## UC03 – Tratar saída indisponível

| | |
|---|---|
| **Atores** | Primário: Operador (é informado do estado da saída e do buffer). Secundário: Sistema Externo de Supervisão (deixa de receber durante a indisponibilidade). |
| **Pré-condição** | Sistema em operação normal. |
| **Gatilho** | A interface de saída sinaliza que o canal ficou indisponível. |
| **Fluxo principal** | 1. A interface de saída sinaliza a indisponibilidade do canal.<br>2. O sistema entra em modo de degradação: suspende a transmissão e continua recebendo, validando e classificando mensagens. [**RQ10**]<br>3. O sistema retém as mensagens que não podem ser transmitidas, até 4 críticas e 16 periódicas. [**RQ04, RQ12**]<br>4. O Operador é informado de que a saída está indisponível.<br>5. A interface de saída sinaliza que o canal voltou.<br>6. O sistema retoma a transmissão, enviando primeiro as mensagens críticas retidas e depois as periódicas. |
| **Fluxos alternativos** | 3a. Uma fila atinge sua capacidade: o sistema sinaliza buffer cheio, descarta a mensagem que acabou de chegar e contabiliza o descarte na estatística. [**RQ05**] |
| **Pós-condição** | As mensagens retidas foram transmitidas e o sistema voltou à operação normal; descartes, se houve, estão registrados. |

## UC04 – Tratar mensagem inválida

| | |
|---|---|
| **Atores** | Sistema Externo de Supervisão (parte interessada: este caso de uso garante que nenhuma mensagem inválida chegue a ela). O descarte é executado pelo próprio sistema (RQ07). A mensagem inválida pode vir de qualquer sensor: ela chega a este caso de uso pelo fluxo alternativo 2a do UC01 e do UC02, onde os sensores já aparecem como atores. |
| **Pré-condição** | Sistema em operação. |
| **Gatilho** | Chegada de uma mensagem com algum campo fora da faixa válida ou ausente (UC01 ou UC02, fluxo 2a). |
| **Fluxo principal** | 1. Um sensor envia uma mensagem.<br>2. O sistema verifica os campos: origem (0 a 2), tipo (0 ou 1) e prioridade (0 ou 1).<br>3. A verificação falha: o sistema descarta a mensagem, que não segue para a classificação nem para a saída. [**RQ07**]<br>4. O sistema registra a recepção inválida na estatística do sensor de origem. [**RQ06**] |
| **Fluxos alternativos** | 4a. O próprio campo de origem é inválido: como os três sensores chegam pela mesma porta, a recepção é contabilizada como de origem desconhecida. |
| **Pós-condição** | Nenhuma mensagem inválida chega à Supervisão; o contador de inválidas foi atualizado. |

# Anexo C Cenários de teste detalhados

## TC01 – Mensagem periódica válida de temperatura

| | |
|---|---|
| **Estado inicial** | Operação normal; filas vazias; canal de saída ativo; limite de temperatura de 45 °C. |
| **Estímulo** | `temp_sensor` envia {origem = 0, tipo = leitura, valor = 25}. |
| **Caminho no modelo** | `Receiver` valida os campos e envia `RxReport`{0, válida} à `Statistics`; `MsgClassifier` lê `temp_limit` = 45 e, como 25 ≤ 45, marca prioridade 0 e envia por `per_out`; `BufferMgr` tem crédito periódico (16 → 15) e encaminha à fila `per_in` do `Arbiter`; o `Arbiter` transmite por `msg_out` e devolve o crédito (15 → 16); `OutputIf` recebe os 8 bytes. |
| **Máquina de estados** | Idle → Receiving → Buffered → Transmitting → Idle. |
| **Resultado esperado** | Mensagem classificada como periódica, armazenada na fila periódica e entregue no formato de 8 bytes; contador do sensor 0 incrementado. |
| **Verificação** | Inspeção: o caminho existe e está conectado (`m_temp`, `m_acq`, `m_per`, `m_tx`). Análise: a validação do OSATE, sem erros, confirma que todas as portas do caminho usam `Msg.impl` e que os tamanhos de dados são coerentes (8 bytes). |

## TC02 – Temperatura e evento crítico simultâneos

| | |
|---|---|
| **Estado inicial** | Operação normal; canal de saída ativo. |
| **Estímulo** | No mesmo instante, `temp_sensor` envia leitura de 25 °C e `motion_sensor` envia evento de movimento. |
| **Caminho no modelo** | As duas mensagens entram na fila de 4 posições do `Receiver`, compartilhada pelos três sensores; o `MsgClassifier` envia o evento por `crit_out` e a leitura por `per_out`; o `BufferMgr` as encaminha às filas correspondentes do `Arbiter`; com as duas filas ocupadas, o despacho do `Arbiter` atende `crit_in` (`Urgency` 2 contra 1) e transmite o evento primeiro. |
| **Resultado esperado** | Evento crítico transmitido antes da leitura periódica; latência do caminho crítico ≤ 100 ms. |
| **Verificação** | Análise: latência de fluxo do OSATE para `e2e_critical` de 91,5 a 95,3 ms no modo SS-PE-ET-FQ-EQL, dentro do limite de 100 ms (no modo AS-PE-DL-EQ-DQL, mais pessimista, 117 ms). Inspeção: `crit_in` com `Urgency` 2 e `per_in` com `Urgency` 1 no `Arbiter`. |

## TC03 – Interface de saída indisponível (exceção)

| | |
|---|---|
| **Estado inicial** | Operação normal; filas vazias. |
| **Estímulo** | `OutputIf` sinaliza `link_down`; em seguida chegam três leituras periódicas e um evento crítico; por fim, `OutputIf` sinaliza `link_up`. |
| **Caminho no modelo** | O `Arbiter` trata `link_down` antes de qualquer mensagem pendente (`Urgency` 3) e suspende a transmissão; `Receiver`, `MsgClassifier` e `BufferMgr` continuam operando; os créditos caem para 3 (críticos) e 13 (periódicos), registrando as mensagens sob guarda do árbitro; com `link_up`, o `Arbiter` transmite primeiro a crítica e depois as três periódicas, devolvendo os créditos. |
| **Máquina de estados** | Buffered → OutputUnavailable (`link_down`) → Buffered (`link_up`) → Transmitting. |
| **Resultado esperado** | Mensagens retidas sem perda durante a indisponibilidade; sistema em OutputUnavailable; na volta do canal, a crítica é a primeira transmitida. |
| **Verificação** | Inspeção: `link_down` e `link_up` conectados ao `Arbiter` (`e_down`, `e_up`) com `Urgency` 3; filas de 4 e 16 posições comportam as quatro mensagens retidas. |

## TC04 – Buffer atinge a capacidade máxima (exceção)

| | |
|---|---|
| **Estado inicial** | Canal de saída indisponível (como no TC03); fila periódica com 16 mensagens; créditos periódicos = 0. |
| **Estímulo** | Chega a 17.ª mensagem periódica. |
| **Caminho no modelo** | Sem crédito periódico, o `BufferMgr` dispara `buffer_full`, que segue para a `Statistics` e para a `OutputIf`, descarta a mensagem recém-chegada e contabiliza o descarte; as 16 mensagens retidas permanecem, e as filas do `Arbiter` não transbordam. |
| **Máquina de estados** | OutputUnavailable → BufferFull (`sem_crédito`). |
| **Observação** | Com as taxas máximas de chegada, a fila periódica (até 4 mensagens/s) enche em pelo menos 4 s, e a crítica (até 12 mensagens/s), em cerca de 0,33 s. |
| **Resultado esperado** | `BufferFull` sinalizado; mensagem mais recente descartada e contabilizada; mensagens retidas preservadas. |
| **Verificação** | Inspeção: `buffer_full` conectado à `Statistics` (fila de 16, `AllItems`) e à `OutputIf` (`e_full_stat`, `e_full_out`); capacidades declaradas (`Queue_Size` 4 e 16) e controladas pelos créditos do `BufferMgr`. |

## TC05 – Mensagem com formato inválido (exceção)

| | |
|---|---|
| **Estado inicial** | Operação normal. |
| **Estímulo** | `light_sensor` envia mensagem com tipo = 7, fora da faixa válida (0 ou 1). |
| **Caminho no modelo** | O `Receiver` detecta o campo inválido e não envia a mensagem por `msg_out`; envia `RxReport`{1, inválida} à `Statistics`, que incrementa as recepções e as inválidas do sensor 1. |
| **Máquina de estados** | Idle → Receiving → Idle. |
| **Verificação** | Inspeção: a porta `report` do `Receiver` está ligada à `Statistics` (`c_rep`), o fluxo `fp` só cobre mensagens válidas e `RxReport.impl` carrega o campo `valid`. |
| **Resultado esperado** | Mensagem descartada; nada chega ao classificador nem à saída; estatística atualizada. |

# Anexo D Diagramas de requisitos

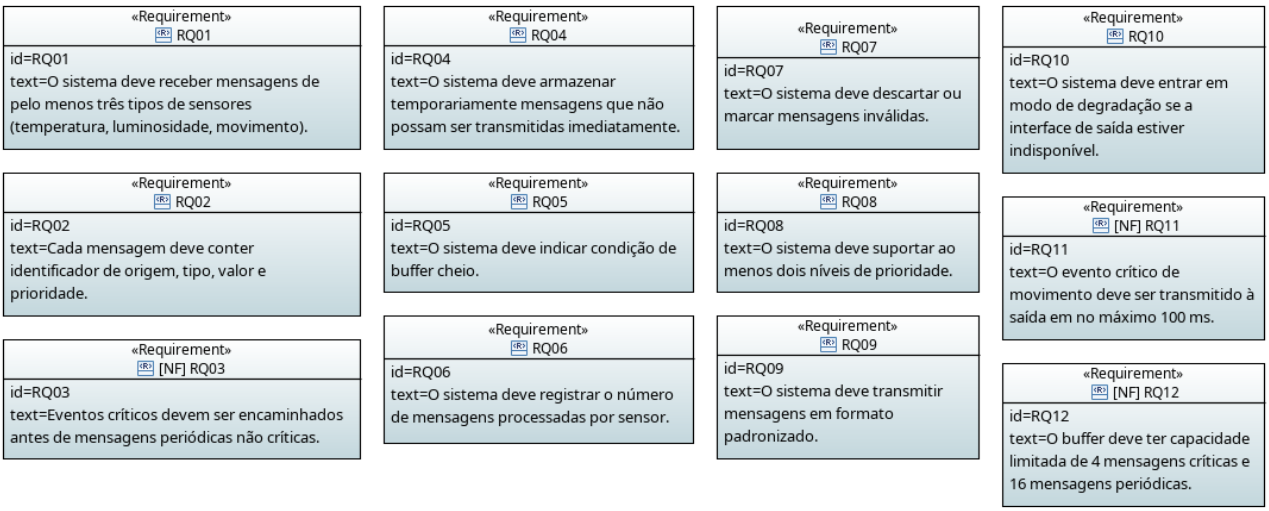



**Figura 7:** Diagrama de requisitos: visão geral (NF indica requisito não funcional).

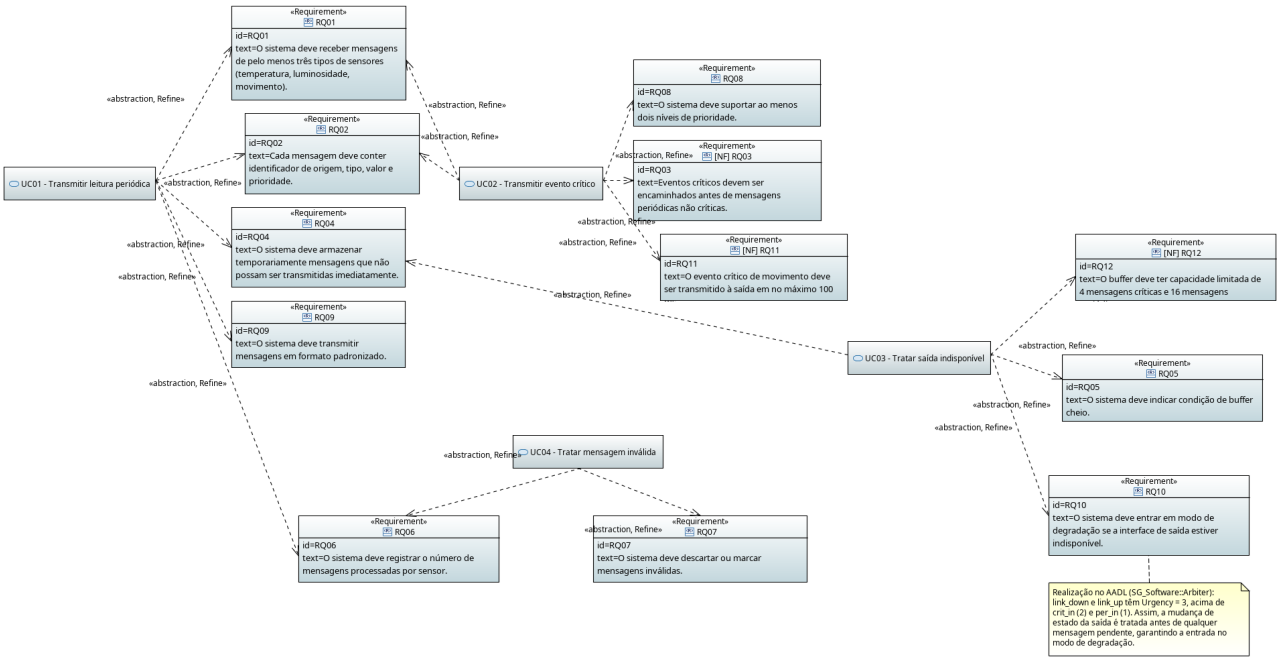



**Figura 8:** Relações «refine» entre casos de uso e requisitos. A nota junto ao RQ10 indica sua realização no AADL.

# Anexo E Código AADL

## SG_Types.aadl

```
-- ============================================================================
--  SG_Types: tipos de dados do SensorGatewaySoC (Projeto 3)
--
--  Pacote-base: não depende de nenhum outro pacote do projeto.
--  É importado pela plataforma e pelo software.
--
--  Dependências do projeto:
--      SG_Types  <-  SG_Platform  <-+
--      SG_Types  <-  SG_Software  <-+-  SG_System
-- ============================================================================
package SG_Types
public
  with Base_Types;

  -- Mensagem padronizada de 8 bytes (RQ02, RQ09).
  -- O campo reservado vem antes de value para alinhar o inteiro no offset 4.
  data Msg
    properties
      Data_Size => 8 Bytes;
  end Msg;

  data implementation Msg.impl
    subcomponents
      source_id : data Base_Types::Unsigned_8;   -- 0 temp, 1 luz, 2 movimento
      msg_type  : data Base_Types::Unsigned_8;   -- 0 leitura, 1 evento
      priority  : data Base_Types::Unsigned_8;   -- 0 periódica, 1 crítica
      reserved  : data Base_Types::Unsigned_8;   -- alinhamento
      value     : data Base_Types::Integer_32;
  end Msg.impl;

  -- Relatório que a recepção manda para a estatística a cada mensagem
  -- recebida: de qual sensor veio e se era válida (RQ06, RQ07).
  data RxReport
    properties
      Data_Size => 2 Bytes;
  end RxReport;

  -- O Boolean de Base_Types não declara tamanho, então o OSATE o conta
  -- como 0 byte e acusa que a soma dos campos (1) é menor que os 2 Bytes
  -- declarados. O tamanho é dado ao próprio campo: 1 + 1 = 2.
  data implementation RxReport.impl
    subcomponents
      source_id : data Base_Types::Unsigned_8;
      valid     : data Base_Types::Boolean { Data_Size => 1 Bytes; };
  end RxReport.impl;

  -- Configuração lida pelo classificador: limite de temperatura acima do
  -- qual a leitura vira crítica (ConfigurationMemory).
  data Config
    properties
      Data_Size => 4 Bytes;
  end Config;

  data implementation Config.impl
    subcomponents
      temp_limit : data Base_Types::Integer_32;   -- em °C, padrão 45
  end Config.impl;

end SG_Types;
```

## SG_Platform.aadl

```
-- ============================================================================
--  SG_Platform: plataforma de execução e dispositivos do SensorGatewaySoC
--
--  O hardware em que o software vai rodar (CPU, memória, barramento) e os
--  dispositivos na fronteira do sistema (sensores e interface de saída).
--  Importa só os tipos de dados: não sabe nada sobre o software.
-- ============================================================================
package SG_Platform
public
  with SG_Types;

  -- ==========================================================================
  --  PLATAFORMA DE EXECUÇÃO
  -- ==========================================================================

  -- Barramento interno do SoC
  bus SocBus
  end SocBus;

```

```
  -- CPU + escalonador de prioridade fixa com preempção, política Rate
  -- Monotonic: é o que dá significado às prioridades das threads.
  -- Se o OSATE marcar erro no valor, apague RMS e use Ctrl+Espaço dentro
  -- dos parênteses: a lista de valores aceitos vem do property set
  -- AADL_Project e muda entre versões.
  processor Cpu
    features
      bus_ic : requires bus access SocBus;
    properties
      Scheduling_Protocol => (RMS);
  end Cpu;

  -- Guarda configuração, filas e contadores (ConfigurationMemory, MessageBuffer)
  memory Ram
    features
      bus_ic : requires bus access SocBus;
    properties
      Memory_Size => 64 KByte;
  end Ram;

  -- ============================================================================
  --  DISPOSITIVOS
  --  Cada device representa o sensor junto com o controlador de interface
  --  mapeado no barramento. O interior não é modelado (caixa-preta).
  -- ============================================================================

  -- Interface comum aos três sensores, herdada pelos tipos abaixo.
  -- O fluxo fim a fim nasce aqui; 1 ms é o tempo do controlador entre a
  -- detecção e a mensagem pronta na porta.
  -- O Deadline de 1 ms repete esse tempo: sem ele, o deadline padrão é o
  -- período (100 ms no sensor de movimento), e a análise de latência no
  -- modo "pior caso = deadline" contaria 100 ms só na origem.
  device Sensor
    features
      msg_out : out event data port SG_Types::Msg.impl;
      bus_ic  : requires bus access SocBus;
    flows
      src : flow source msg_out { Latency => 0 ms .. 1 ms; };
    properties
      Deadline => 1 ms;
  end Sensor;

  device TempSensor extends Sensor
    properties
      Dispatch_Protocol => Periodic;
      Period            => 500 ms;
  end TempSensor;

  device LightSensor extends Sensor
    properties
      Dispatch_Protocol => Periodic;
      Period            => 500 ms;
  end LightSensor;

  -- Movimento só envia quando detecta: esporádico, intervalo mínimo de 100 ms.
  -- Hipótese sobre o ambiente: o controlador do sensor nunca envia mais de
  -- um evento a cada 100 ms. Toda a conta de taxa de chegada depende disso.
  device MotionSensor extends Sensor
    properties
      Dispatch_Protocol => Sporadic;
      Period            => 100 ms;
  end MotionSensor;

  -- Interface de saída (UART, CAN ou rádio; não implementada).
  -- O fluxo fim a fim termina aqui; 1 ms cobre a transmissão de 8 bytes
  -- numa UART a 115200 baud (8 x 10 bits / 115200 bit/s = 0,7 ms).
  device OutputIf
    features
      msg_in      : in event data port SG_Types::Msg.impl;  -- a transmitir
      buffer_full : in event port;                          -- estado (RQ05)
      link_down   : out event port;                         -- saída caiu (RQ10)
      link_up     : out event port;                         -- saída voltou (RQ10)
      bus_ic      : requires bus access SocBus;
    flows
      snk : flow sink msg_in { Latency => 0 ms .. 1 ms; };
  end OutputIf;

end SG_Platform;
```

## SG_Software.aadl

```
-- ============================================================================
--  SG_Software: software de aplicação do SensorGatewaySoC
--
--  As cinco threads e os três processos que as agrupam.
--
--  Repare que este pacote importa SG_Types, mas NÃO importa SG_Platform:
```

```
--  o software não sabe em que hardware vai rodar. Quem decide isso é só o
--  SG_System, pelos bindings. O mesmo software poderia ser alocado em outra
--  plataforma trocando apenas o pacote do sistema.
--
--  Tempos em us: o tipo Time do AADL é inteiro, então 0,2 ms se escreve
--  200 us. Prioridades por Rate Monotonic (maior número = mais prioritária).
-- ============================================================================
package SG_Software
public
  with SG_Types;

  -- ##########################################################################
  --  THREADS
  -- ##########################################################################

  -- SensorInterface: atende os três sensores, valida e descarta as inválidas
  -- (RQ01, RQ07) e informa cada recepção à estatística (RQ06).
  thread Receiver
    features
      msg_in  : in event data port SG_Types::Msg.impl { Queue_Size => 4; };
      msg_out : out event data port SG_Types::Msg.impl;       -- só as válidas
      report  : out event data port SG_Types::RxReport.impl;  -- para a estatística
    flows
      fp : flow path msg_in -> msg_out;
    properties
      Dispatch_Protocol      => Sporadic;
      Period                 => 5 ms;
      Compute_Execution_Time => 200 us .. 500 us;
      Deadline               => 5 ms;
      Priority               => 5;
  end Receiver;

  -- StatisticsManager: contadores por sensor e de buffer cheio (RQ06).
  -- Periódica e de baixa prioridade: a cada segundo processa todos os
  -- relatórios acumulados de uma vez (AllItems). A fila de 16 cobre a taxa
  -- máxima de chegada (2 + 2 + 10 = 14 mensagens/s).
  -- buffer_full também precisa de fila e de AllItems: sem eles, a fila tem
  -- 1 posição e a thread retira 1 evento por despacho, então vários
  -- "buffer cheio" no mesmo segundo seriam contados como um só.
  thread Statistics
    features
      report_in   : in event data port SG_Types::RxReport.impl {
        Queue_Size       => 16;
        Dequeue_Protocol => AllItems;
      };
      buffer_full : in event port {
        Queue_Size       => 16;
        Dequeue_Protocol => AllItems;
      };
    properties
      Dispatch_Protocol      => Periodic;
      Period                 => 1000 ms;
      Compute_Execution_Time => 500 us .. 1 ms;
      Deadline               => 1000 ms;
      Priority               => 1;
  end Statistics;

  -- MessageFormatter: classifica em crítica ou periódica, preenche o campo
  -- priority (RQ08) e manda cada mensagem para a fila certa. Lê o limite de
  -- temperatura da configuração, só para leitura.
  -- (Nome MsgClassifier porque "classifier" é palavra reservada do AADL.)
  thread MsgClassifier
    features
      msg_in   : in event data port SG_Types::Msg.impl { Queue_Size => 4; };
      crit_out : out event data port SG_Types::Msg.impl;
      per_out  : out event data port SG_Types::Msg.impl;
      cfg      : requires data access SG_Types::Config.impl {
        Access_Right => read_only;
      };
    flows
      fp_crit : flow path msg_in -> crit_out;
      fp_per  : flow path msg_in -> per_out;
    properties
      Dispatch_Protocol      => Sporadic;
      Period                 => 5 ms;
      Compute_Execution_Time => 200 us .. 500 us;
      Deadline               => 5 ms;
      Priority               => 4;
  end MsgClassifier;

  -- MessageBuffer (controle): encaminha cada mensagem para a fila certa do
  -- árbitro e controla a ocupação por CRÉDITOS: começa com 4 créditos
  -- críticos e 16 periódicos, gasta um a cada mensagem enviada e recebe um
  -- de volta a cada mensagem que o árbitro transmite. Isso é necessário
  -- porque uma thread não enxerga a ocupação da fila de outra thread.
  -- Sem créditos numa fila = fila cheia = sinaliza BufferFull (RQ05).
  --
  -- Política de descarte (TC04): sem crédito, a mensagem que acabou de
  -- chegar é descartada, contada e BufferFull é sinalizado. Por isso as
  -- filas do árbitro nunca transbordam; o descarte acontece aqui.
  -- A fila de trânsito crítica usa Error para que, se ela própria encher,
```

```
98  -- uma mensagem crítica nunca suma em silêncio (o padrão seria DropOldest).
99  thread BufferMgr
100   features
101     crit_in     : in event data port SG_Types::Msg.impl {              -- trânsito
102       Queue_Size                => 2;
103       Overflow_Handling_Protocol => Error;
104     };
105     per_in      : in event data port SG_Types::Msg.impl { Queue_Size => 2; };  -- trânsito
106     crit_out    : out event data port SG_Types::Msg.impl;
107     per_out     : out event data port SG_Types::Msg.impl;
108     crit_credit : in event port { Queue_Size => 4; };
109     per_credit  : in event port { Queue_Size => 16; };
110     buffer_full : out event port;
111   flows
112     fp_crit : flow path crit_in -> crit_out;
113     fp_per  : flow path per_in -> per_out;
114   properties
115     Dispatch_Protocol      => Sporadic;
116     Period                 => 5 ms;
117     Compute_Execution_Time => 100 us .. 300 us;
118     Deadline               => 5 ms;
119     Priority               => 3;
120 end BufferMgr;
121
122 -- PriorityArbiter: as duas portas de entrada SÃO as filas do buffer
123 -- (RQ04, RQ12). Cada disparo transmite uma mensagem: se as duas filas têm
124 -- mensagens, a Urgency maior escolhe a crítica (prioridade estrita, RQ03).
125 -- Esporádica com intervalo mínimo de 10 ms = no máximo 100 msg/s na saída.
126 -- Com o enlace caído (link_down) não transmite e retém as mensagens até
127 -- link_up (RQ10). A capacidade continua garantida pelos créditos do
128 -- BufferMgr, que contam as mensagens sob guarda do árbitro.
129 -- Com os créditos funcionando estas filas nunca transbordam: as políticas
130 -- Error e DropOldest abaixo são rede de segurança, não a política normal.
131 -- link_down/link_up têm a maior Urgency para que uma mudança de estado do
132 -- enlace seja tratada antes de qualquer mensagem pendente.
133 thread Arbiter
134   features
135     crit_in : in event data port SG_Types::Msg.impl {
136       Queue_Size                => 4;
137       Overflow_Handling_Protocol => Error;       -- crítica nunca some calada
138       Urgency                   => 2;
139     };
140     per_in : in event data port SG_Types::Msg.impl {
141       Queue_Size                => 16;
142       Overflow_Handling_Protocol => DropOldest;  -- leitura nova vale mais
143       Urgency                   => 1;
144     };
145     msg_out     : out event data port SG_Types::Msg.impl;
146     crit_credit : out event port;   -- devolve crédito a cada crítica transmitida
147     per_credit  : out event port;   -- devolve crédito a cada periódica transmitida
148     link_down   : in event port { Urgency => 3; };
149     link_up     : in event port { Urgency => 3; };
150   flows
151     fp_crit : flow path crit_in -> msg_out;
152     fp_per  : flow path per_in -> msg_out;
153   properties
154     Dispatch_Protocol      => Sporadic;
155     Period                 => 10 ms;
156     Compute_Execution_Time => 1 ms .. 2 ms;
157     Deadline               => 10 ms;
158     Priority               => 2;
159 end Arbiter;
160
161 -- ############################################################################
162 --  PROCESSOS
163 --  Cada processo é um espaço de endereçamento que agrupa threads. As
164 --  features do processo são as "bordas" por onde as conexões entram e saem.
165 -- ############################################################################
166
167 -- Aquisição e validação: recepção + estatística
168 process Acquisition
169   features
170     msg_in      : in event data port SG_Types::Msg.impl;   -- dos três sensores
171     msg_out     : out event data port SG_Types::Msg.impl;
172     buffer_full : in event port;                           -- vem do BufferMgr
173   flows
174     fp : flow path msg_in -> msg_out;
175 end Acquisition;
176
177 process implementation Acquisition.impl
178   subcomponents
179     receiver   : thread Receiver;
180     statistics : thread Statistics;
181   connections
182     c_in   : port msg_in -> receiver.msg_in;               -- desce
183     c_out  : port receiver.msg_out -> msg_out;             -- sobe
184     c_rep  : port receiver.report -> statistics.report_in; -- entre irmãs
185     c_full : port buffer_full -> statistics.buffer_full;   -- desce
186   flows
187     fp : flow path msg_in -> c_in -> receiver.fp -> c_out -> msg_out;
188 end Acquisition.impl;
```

```
189
190   -- Classificação e armazenamento: classificador + gerenciador do buffer,
191   -- mais o dado de configuração que o classificador lê
192   process Classification
193     features
194       msg_in      : in event data port SG_Types::Msg.impl;
195       crit_out    : out event data port SG_Types::Msg.impl;
196       per_out     : out event data port SG_Types::Msg.impl;
197       crit_credit : in event port;
198       per_credit  : in event port;
199       buffer_full : out event port;
200     flows
201       fp_crit : flow path msg_in -> crit_out;
202       fp_per  : flow path msg_in -> per_out;
203   end Classification;
204
205   process implementation Classification.impl
206     subcomponents
207       msg_classifier : thread MsgClassifier;
208       buffer_mgr     : thread BufferMgr;
209       config         : data SG_Types::Config.impl;
210     connections
211       c_in     : port msg_in -> msg_classifier.msg_in;
212       c_crit   : port msg_classifier.crit_out -> buffer_mgr.crit_in;
213       c_per    : port msg_classifier.per_out -> buffer_mgr.per_in;
214       c_crit_o : port buffer_mgr.crit_out -> crit_out;
215       c_per_o  : port buffer_mgr.per_out -> per_out;
216       c_ccred  : port crit_credit -> buffer_mgr.crit_credit;
217       c_pcred  : port per_credit -> buffer_mgr.per_credit;
218       c_full   : port buffer_mgr.buffer_full -> buffer_full;
219       -- Acesso à configuração: com seta de sentido único, porque o
220       -- classificador só lê (Access_Right => read_only na feature)
221       a_cfg    : data access config -> msg_classifier.cfg;
222     flows
223       fp_crit : flow path msg_in -> c_in -> msg_classifier.fp_crit -> c_crit
224                 -> buffer_mgr.fp_crit -> c_crit_o -> crit_out;
225       fp_per  : flow path msg_in -> c_in -> msg_classifier.fp_per -> c_per
226                 -> buffer_mgr.fp_per -> c_per_o -> per_out;
227   end Classification.impl;
228
229   -- Transmissão: árbitro (dono das filas) + controle do enlace
230   process Transmission
231     features
232       crit_in     : in event data port SG_Types::Msg.impl;
233       per_in      : in event data port SG_Types::Msg.impl;
234       msg_out     : out event data port SG_Types::Msg.impl;
235       crit_credit : out event port;
236       per_credit  : out event port;
237       link_down   : in event port;
238       link_up     : in event port;
239     flows
240       fp_crit : flow path crit_in -> msg_out;
241       fp_per  : flow path per_in -> msg_out;
242   end Transmission;
243
244   process implementation Transmission.impl
245     subcomponents
246       arbiter : thread Arbiter;
247     connections
248       c_crit  : port crit_in -> arbiter.crit_in;
249       c_per   : port per_in -> arbiter.per_in;
250       c_out   : port arbiter.msg_out -> msg_out;
251       c_ccred : port arbiter.crit_credit -> crit_credit;
252       c_pcred : port arbiter.per_credit -> per_credit;
253       c_down  : port link_down -> arbiter.link_down;
254       c_up    : port link_up -> arbiter.link_up;
255     flows
256       fp_crit : flow path crit_in -> c_crit -> arbiter.fp_crit -> c_out -> msg_out;
257       fp_per  : flow path per_in -> c_per -> arbiter.fp_per -> c_out -> msg_out;
258   end Transmission.impl;
259
260 end SG_Software;
```

## SG_System.aadl

```
1  -- ============================================================================
2  --  SG_System: integração do SensorGatewaySoC (Projeto 3)
3  --
4  --  O único pacote que conhece ao mesmo tempo a plataforma e o software.
5  --  Aqui acontecem as três decisões de integração:
6  --    1. quais componentes existem (subcomponents)
7  --    2. como se ligam (connections)
8  --    3. onde cada software roda e mora (bindings)
9  --  e o requisito de latência fim a fim (flows).
10 --
11 --  Raiz para instanciar: SG_System::SensorGatewaySoC.impl
12 -- ============================================================================
```

```
package SG_System
public
  with SG_Platform, SG_Software;

  system SensorGatewaySoC
  end SensorGatewaySoC;

  system implementation SensorGatewaySoC.impl
    subcomponents
      -- Plataforma
      cpu            : processor SG_Platform::Cpu;
      ram            : memory SG_Platform::Ram;
      soc_bus        : bus SG_Platform::SocBus;
      -- Dispositivos
      temp_sensor    : device SG_Platform::TempSensor;
      light_sensor   : device SG_Platform::LightSensor;
      motion_sensor  : device SG_Platform::MotionSensor;
      out_if         : device SG_Platform::OutputIf;
      -- Software
      acquisition    : process SG_Software::Acquisition.impl;
      classification : process SG_Software::Classification.impl;
      transmission   : process SG_Software::Transmission.impl;
    connections
      -- Topologia física: tudo pendurado no barramento do SoC
      hw_cpu    : bus access soc_bus <-> cpu.bus_ic;
      hw_ram    : bus access soc_bus <-> ram.bus_ic;
      hw_temp   : bus access soc_bus <-> temp_sensor.bus_ic;
      hw_light  : bus access soc_bus <-> light_sensor.bus_ic;
      hw_motion : bus access soc_bus <-> motion_sensor.bus_ic;
      hw_out    : bus access soc_bus <-> out_if.bus_ic;

      -- Sensores -> aquisição: três fontes na mesma porta (fan-in)
      m_temp   : port temp_sensor.msg_out -> acquisition.msg_in;
      m_light  : port light_sensor.msg_out -> acquisition.msg_in;
      m_motion : port motion_sensor.msg_out -> acquisition.msg_in;

      -- Pipeline de mensagens
      m_acq  : port acquisition.msg_out -> classification.msg_in;
      m_crit : port classification.crit_out -> transmission.crit_in;
      m_per  : port classification.per_out -> transmission.per_in;
      m_tx   : port transmission.msg_out -> out_if.msg_in;

      -- Créditos: o árbitro devolve ao gerenciador do buffer
      e_ccred : port transmission.crit_credit -> classification.crit_credit;
      e_pcred : port transmission.per_credit -> classification.per_credit;

      -- BufferFull: uma saída, dois destinos (fan-out)
      e_full_stat : port classification.buffer_full -> acquisition.buffer_full;
      e_full_out  : port classification.buffer_full -> out_if.buffer_full;

      -- Estado do enlace
      e_down : port out_if.link_down -> transmission.link_down;
      e_up   : port out_if.link_up -> transmission.link_up;

    -- O caminho do evento crítico, do sensor de movimento até a saída,
    -- sempre alternando fluxo de componente e conexão (RQ03, RQ11).
    flows
      e2e_critical : end to end flow
          motion_sensor.src
          -> m_motion -> acquisition.fp
          -> m_acq    -> classification.fp_crit
          -> m_crit   -> transmission.fp_crit
          -> m_tx     -> out_if.snk
        { Latency => 0 ms .. 100 ms; };

    properties
      -- Binding: onde o software roda e onde mora
      Actual_Processor_Binding => (reference (cpu))
        applies to acquisition, classification, transmission;
      Actual_Memory_Binding => (reference (ram))
        applies to acquisition, classification, transmission;
      -- Só as conexões que envolvem dispositivos passam pelo barramento.
      -- Entre processos na mesma CPU a comunicação é pela memória.
      Actual_Connection_Binding => (reference (soc_bus))
        applies to m_temp, m_light, m_motion, m_tx, e_down, e_up, e_full_out;
  end SensorGatewaySoC.impl;

end SG_System;
```